

Learning Procedural World Generation

Tyler Forgione

September 3, 2026

Contents

1	Introduction	1
2	Noise Generation	1
2.1	White Noise	1
2.2	Value Noise	2
2.3	Fractal Noise	3
3	Terrain Generation	3
3.1	Heightmaps	3
4	Experiments	3
5	Notes and Future Work	3

1 Introduction

This document records my progress while learning procedural world generation, including the algorithms, design decisions, experiments, and results developed throughout the project.

2 Noise Generation

A common way to generate a 2D map or terrain is to use noise. We often think of noise as a random signal. We think of static on a Television screen when the cable goes out. That is a very specific kind of noise, called white noise.

2.1 White Noise

In 2D, white noise is a grid in which each cell has a random intensity, generally between 0 and 1. The result is a mess of white and black dots on a map that means little. We can create white noise very easily by randomly sampling values in $[0, 1]$ and assigning them to cells on a grid. Below is a Python implementation of white noise using NumPy:

```
def white_noise(height, width, seed):  
    rng = np.random.default_rng(seed)  
    return rng.random(size=(height, width), dtype=np.float32)
```

This results in an output like that in Figure 1a. As is obvious, this is completely random! There is no meaning behind the colour of any dot. If we were to map this in 3D, we'd get mountains with essentially 90 deg angles with the ground below them. There would be no slope to any hill, mountain, or valley. To show this, I've plotted the exact same noise map in 3D. Figure 1b shows the terrain we'd get if only using white noise in our generation.

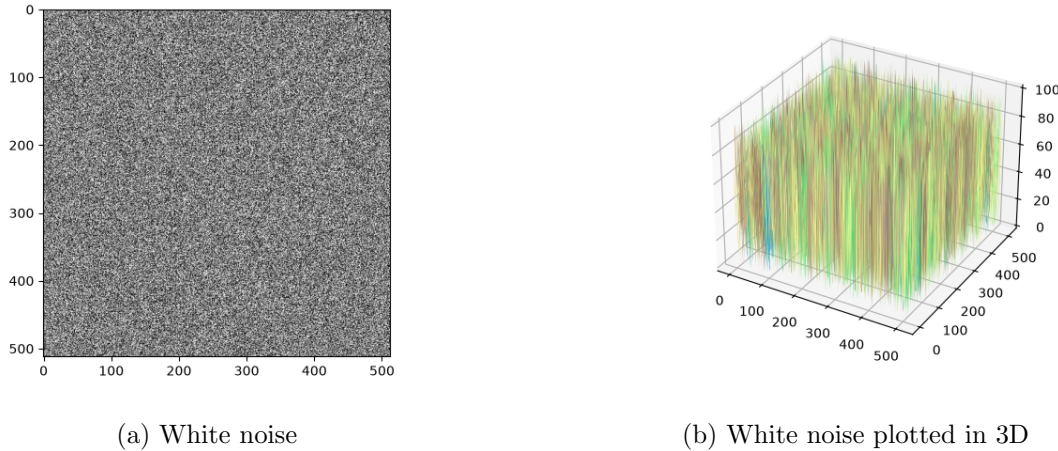


Figure 1: White noise in 2D and 3D

Since terrain needs some sort of synchronicity between adjacent points, white noise won't cut it. Luckily, there are other noise algorithms that provide far better results.

2.2 Value Noise

The idea behind value noise is simple: create a grid of random values at integer points (x, y) and interpolate between them to achieve smooth transitions. Essentially, for any integer point, we must find the surrounding lattice points (4 corners of the cell) and interpolate between them.

Just doing this, however, does not achieve the smoothest of transitions. Without using smoothing, we still may get clunky or jagged edges. Thus, the final piece of the puzzle for value noise is an additional smoothing on the weight parameter in linear interpolation.

Mathematically it's this: we begin with a smoothing function, the cubic Hermite smoothstep

$$S(t) = 3t^2 - 2t^3 \tag{1}$$

We can then define the following:

$$s = S(u) \qquad t = S(v)$$

Next, we can use our linear interpolation function

$$L(a, b, t) = a + t(b - a) \tag{2}$$

to interpolate the top two points and bottom two points horizontally

$$\begin{aligned} T &= L(v_{00}, v_{10}, s) \\ B &= L(v_{01}, v_{11}, s) \end{aligned}$$

and to interpolate these two vertically

$$N(x, y) = L(T, B, t)$$

Substituting and expanding gets us

$$N(x, y) = L(L(v_{00}, v_{10}, S(u)), L(v_{01}, v_{11}, S(u)), S(v)) \quad (3)$$

Where S and L are equation 1 and equation 2, respectively.

One other factor we may want to take into account is the wavelength, which is the distance between adjacent lattice points. The lower the wavelength, the higher the spatial frequency of the noise. As wavelength goes to 1, the closer we get to white noise because there will be less interpolation between lattice points.

From equation 3, we can replace x and y with $x' = \lfloor \frac{x}{\lambda} \rfloor$ and $y' = \lfloor \frac{y}{\lambda} \rfloor$, where λ is the wavelength.

That just about wraps up value noise. The only other possible change is to use octaves and summing up noise from different octaves to create a more detailed terrain.

2.3 Fractal Noise

3 Terrain Generation

3.1 Heightmaps

4 Experiments

5 Notes and Future Work